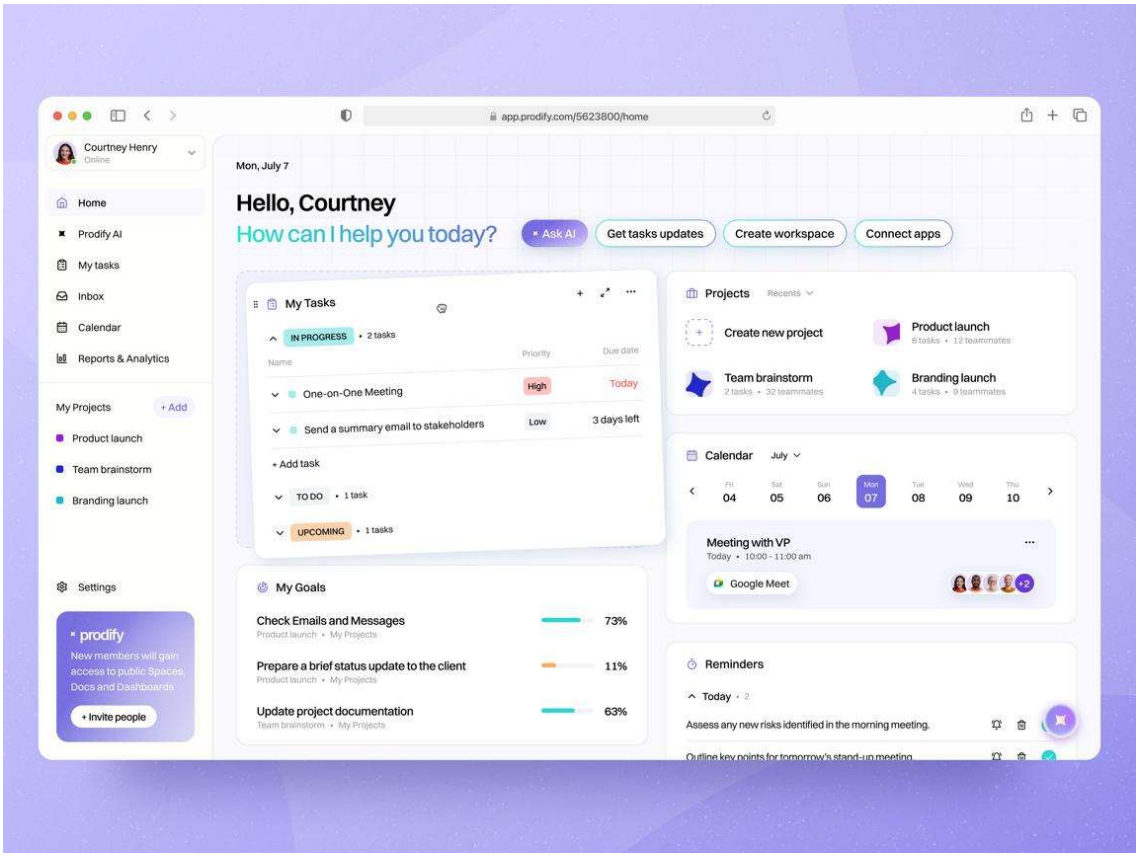
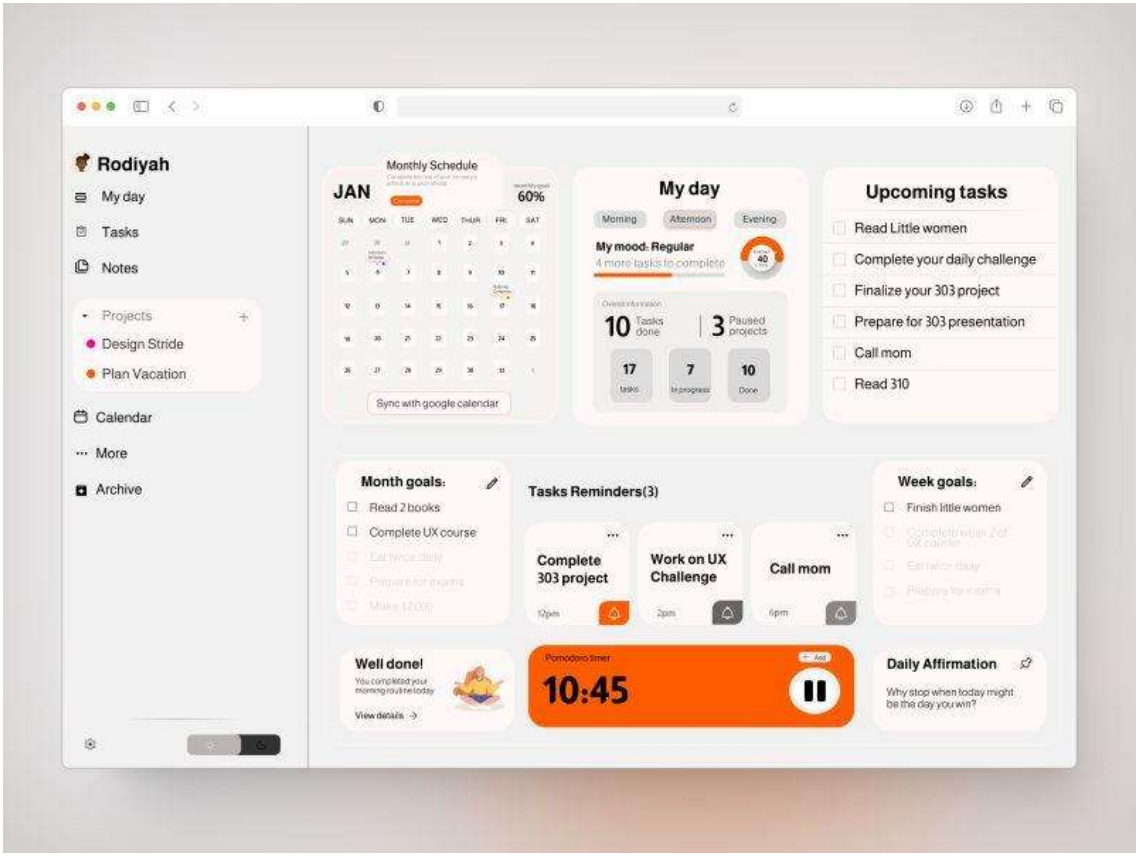
Intelligent Personal Assistant – JavaScript Project for 9-Hour Hands-on Training

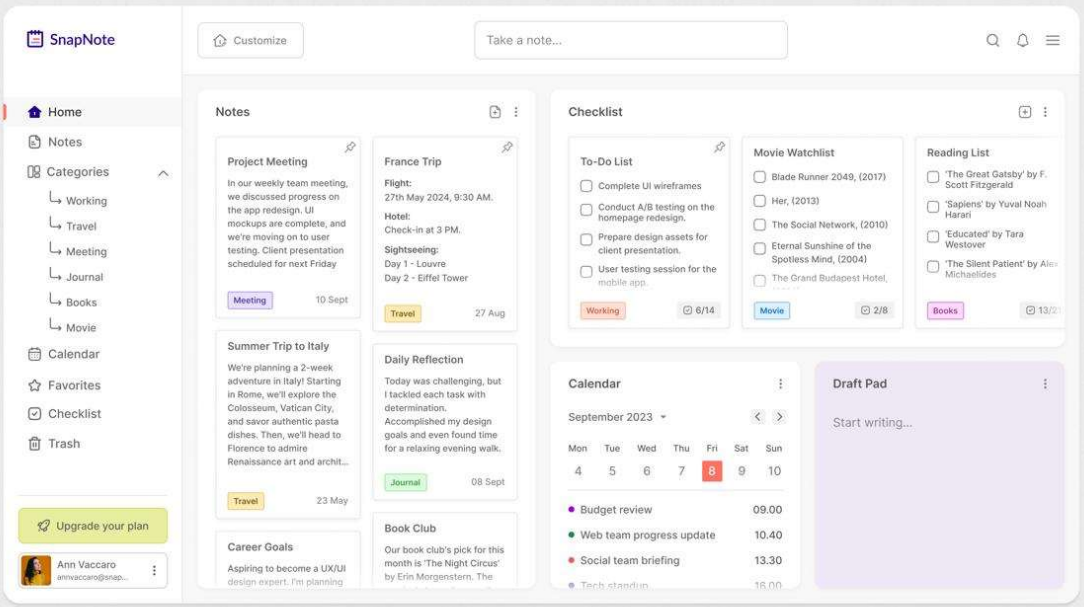
For your 2nd-year UG Computer Science students, I recommend developing a web-based Intelligent Personal Assistant using modern JavaScript (ECMAScript 6+). Students will build the application step by step, learning each JavaScript concept while adding a working feature to the project.

The goal is to finish the nine-hour training with a functional project, complete source code, and a clear understanding of how modern JavaScript is used in real-world application development.

1. Project overview




Rodiyah
My day
Tasks
Notes
Projects
Design Stride
Plan Vacation
Calendar
More
Archive
Monthly Schedule
JAN
60%
Sync with google calendar
My day
Morning
Afternoon
Evening
My mood: Regular
4 more tasks to complete
10 Tasks done
3 Paused projects
17 tasks
7 In progress
10 Done
Upcoming tasks
Read Little women
Complete your daily challenge
Finalize your 303 project
Prepare for 303 presentation
Call mom
Read 310
Month goals:
Read 2 books
Complete UX course
Tasks Reminders(3)
Complete 303 project
12pm
Work on UX Challenge
2pm
Call mom
6pm
Week goals:
Finish little women
Well done!
View details
Pomodoro timer
10:45
Daily Affirmation
Why stop when today might be the day you win?


SnapNote
Customize
Take a note...
Home
Notes
Categories
Working
Travel
Meeting
Journal
Books
Movie
Calendar
Favorites
Checklist
Trash
Upgrade your plan
Ann Vaccaro
Notes
Project Meeting
In our weekly team meeting, we discussed progress on the app redesign, UI mockups are complete, and we're moving on to user testing. Client presentation scheduled for next Friday
Meeting
10 Sept
France Trip
Flight:
27th May 2024, 9:30 AM.
Hotel:
Check-in at 3 PM.
Sightseeing:
Day 1 - Louvre
Day 2 - Eiffel Tower
Travel
27 Aug
Summer Trip to Italy
We're planning a 2-week adventure in Italy! Starting in Rome, we'll explore the Colosseum, Vatican City, and savor authentic pasta dishes. Then, we'll head to Florence to admire Renaissance art and archit...
Travel
23 May
Daily Reflection
Today was challenging, but I tackled each task with determination. Accomplished my design goals and even found time for a relaxing evening walk.
Journal
08 Sept
Book Club
Our book club's pick for this month is 'The Night Circus' by Erin Morgenstern. The
Career Goals
Aspiring to become a UX/UI design expert. I'm planning
Checklist
To-Do List
Complete UI wireframes
Conduct A/B testing on the homepage redesign.
Prepare design assets for client presentation.
User testing session for the mobile app.
Working
6/14
Movie Watchlist
Blade Runner 2049, (2017)
Her, (2013)
The Social Network, (2010)
Eternal Sunshine of the Spotless Mind, (2004)
The Grand Budapest Hotel,
Movie
2/8
Reading List
'The Great Gatsby' by F. Scott Fitzgerald
'Sapiens' by Yuval Noah Harari
'Educated' by Tara Westover
'The Silent Patient' by Alex Michaelides
Books
Calendar
September 2023
Mon Tue Wed Thu Fri Sat Sun
4 5 6 7 8 9 10
Budget review 09.00
Web team progress update 10.40
Social team briefing 13.30
Draft Pad
Start writing...

11

Project title: Intelligent Personal Assistant (IPA)

Technology stack: HTML5, CSS3, modern JavaScript (ES modules), browser Local Storage and the Fetch API.

The project will run directly in a web browser. Students will not need to install a database or configure a backend server, making it suitable for a college computer laboratory.

Features students will develop

1. Daily dashboard

Display today's date, task statistics, pending tasks, completed tasks and a daily summary.

2. Task manager

Add, edit, complete, delete and filter daily activities by priority and status.

3. Notes and reminders

Save personal notes, record important information and create reminders with dates.

4. Daily expense tracker

Record expenses, categorize transactions and calculate total spending.

5. Assistant suggestions

Provide rule-based suggestions, such as highlighting overdue tasks, identifying high-priority activities and summarizing daily progress.

Teaching note: The assistant's intelligence will initially come from JavaScript rules and data analysis rather than an external AI service. This keeps the project accessible, inexpensive and appropriate for introductory training.

2. Download the complete project source code

Intelligent Personal Assistant

JavaScript ES6+ · HTML · CSS · ES modules

10 source and documentation files

Includes the working application, separate JavaScript modules, Local Storage persistence, responsive styling, rule-based suggestions and an instructor-oriented README with the nine-hour schedule.

Download complete source code (ZIP)

Project folder structure

```
intelligent-personal-assistant/
|
├── index.html
├── styles.css
├── README.md
|
└── js/
    ├── main.js
    ├── storage.js
    ├── utils.js
    ├── task-manager.js
    ├── note-manager.js
    ├── expense-manager.js
    └── assistant.js
```

Each file has a specific responsibility, so students can learn modular JavaScript rather than writing the entire application in one large script.

3. How to run the project

The application uses JavaScript ES modules, so run it through a local web server.

1. Download and extract the ZIP file.
2. Open the extracted folder in Visual Studio Code.
3. Install the Live Server extension if needed.
4. Right-click index.html and select Open with Live Server.
5. Try adding tasks, completing them, saving notes and recording expenses. Refresh the browser to verify that the records persist.

Alternatively, if Python is installed, run this command from the project directory:

```
python -m http.server 8000
```

Open http://localhost:8000 in your browser.

No npm packages or database setup are required. The core features run locally; an optional Fetch API example demonstrates network requests and error handling.

4. Detailed nine-hour training plan

I recommend the following schedule, with each hour ending in a working feature or a small coding exercise.

Hour 1

60 min

Introduction to ECMAScript, variables and scoping

- ECMAScript and JavaScript; ES6+ overview
- var, let, const, block scope and module scope
- Primitive types, arrays and objects

Practical: Create sample task records and inspect their values.

Hour 2

60 min

Arrow functions and template literals

- Function declarations versus arrow functions
- Parameters, return values and callbacks
- Template strings using backticks and ${...}

Practical: Implement the personalized greeting and dynamic task messages.

Hour 3

60 min

Destructuring, default, rest and spread operators

- Extract values from objects and arrays
- Default parameters and default values
- Rest parameters and spread syntax
- Immutable object updates

Practical: Update task objects and calculate expense totals.

Hour 4

60 min

Enhanced object literals

- Property shorthand
- Method shorthand
- Computed property names

Practical: Create and update structured activity, note and expense objects.

Hour 5

60 min

Classes and object-oriented JavaScript

- Classes, instances, constructors and methods
- Encapsulation and private fields
- Validation and data management

Practical: Develop the TaskManager class.

Hour 6

60 min

JavaScript modules

- Named exports and imports
- Separation of concerns
- Module scope and dependencies

Practical: Separate task, notes, expenses, utilities and storage into modules.

Hour 7

60 min

DOM, event handling and persistence

- Form submission and event listeners
- Dynamic HTML rendering
- CRUD operations
- Local Storage and JSON serialization

Practical: Connect the modules to the dashboard and persist user records.

Hour 8

60 min

Promises and asynchronous programming

- Synchronous versus asynchronous execution
- Promises and .then() / .catch()
- async / await
- Fetch API, JSON and error handling

Practical: Examine the optional API request and implement a loading/error-state exercise.

Hour 9

60 min

Integration, testing and presentation

- Debug with browser DevTools
- Test validation and persistence
- Explain the project architecture
- Add one student-selected enhancement

Practical: Demonstrate the completed application and submit the source code.

5. Suggested teaching method

Rather than giving students all the source code at the beginning, use a build–explain–extend approach:

1. Build: Introduce the next feature and provide the relevant starter code.
2. Explain: Identify the JavaScript concepts used in that feature.
3. Extend: Ask students to modify the code or add a small enhancement.
4. Test: Verify the result in the browser and discuss any errors.

For example, after teaching classes, students can develop TaskManager. After teaching modules, they can split the application into separate files. When learning asynchronous programming, they can inspect how the Fetch API handles a response and what happens when the network is unavailable.

6. Student assessment activities

Assessment task	**Marks**

Explain let, const, scope and arrow functions	10
Demonstrate destructuring, rest and spread	10
Implement a new task-management feature	20
Explain classes and module imports/exports	15
Demonstrate Promise and async/await error handling	15
Test the completed application and explain its architecture	30
Total	100

Possible extension assignments include a weekly expense report, task search, JSON/CSV export, a habit tracker or browser notifications.

My recommendation: Use the provided ZIP as the instructor's reference implementation and reveal the relevant files progressively during the workshop. The project is intentionally browser-based, so students can concentrate on JavaScript concepts without losing training time to backend configuration. One important distinction to explain is that the current assistant uses rule-based suggestions; integrating a real AI model would be a separate advanced exercise requiring secure API-key handling and typically a backend.